AGRICULTURE FARM MANAGEMENT SYSTEM

(AFMSystem)

Project Documentation

Spring Boot · MySQL · Spring Security · JWT

1. Project Overview

The Agriculture Farm Management System (AFMSystem) is a backend REST API built with Java and Spring Boot. It gives farmers one secure place to manage their farms, fields, crops, daily farm activities, expenses and harvests, and to see how much profit each farm makes. MySQL stores the data, while Spring Security and JWT make sure only logged-in users can reach it.

Every farmer sees only their own data. An ADMIN role manages users and has access to everything. Access tokens are short-lived and are renewed with refresh tokens.

It follows the same layered design as the Online Examination System: Controller → Service → Repository → Database, with DTOs for input/output and a global exception handler.

2. Objectives

- Provide secure registration and login using JWT access tokens and rotating refresh tokens.
- Link every login account to a Farmer profile and limit each farmer to their own data.
- Provide an ADMIN role that can manage users (promote, deactivate, link profiles) and access all data.
- Divide each farm into fields and record the crops grown in them.
- Log farm activities such as plowing, sowing, irrigation, fertilizing, weeding and pest control.
- Track farm expenses by category (seed, fertilizer, labour, equipment and more).
- Record harvests and calculate revenue automatically.
- Generate expense, harvest and profit reports for any date range (season).
- Provide pagination, sorting, validation and centralized exception handling.
- Provide automated unit and integration tests, and Swagger/OpenAPI documentation.
- Keep secrets out of the code, with a production profile that refuses to start without them.

3. User Roles

AFMSystem has two roles. Registration always creates a FARMER; the role cannot be chosen in the request. The first ADMIN account is created at startup from the ADMIN_EMAIL and ADMIN_PASSWORD environment variables, and later users can be promoted by an ADMIN.

Role	Main responsibilities
FARMER	Register, log in, and create, view and update their own farms, fields, crops, activities, expenses and harvests. View their own reports and profit. Cannot delete records, list all farmers or manage users.
ADMIN	Access to all data. Delete any record, list all farmers, and manage users (list, change role, activate or deactivate, link a farmer profile).

4. Technologies

Technology	Use
Java 21	Application development
Spring Boot 3.5.5	Backend REST application
Spring Data JPA / Hibernate	Database ORM, tables auto-created with ddl-auto=update
MySQL	Database (agriculture_farm_db)
Spring Security	Authentication and role-based authorization
JWT (jjwt 0.12.6)	Stateless access tokens
Refresh tokens (SHA-256 hashed)	Renew access tokens without logging in again
BCrypt	Password hashing
Bean Validation	Request validation with @Valid
Lombok	Reduces boilerplate in entity classes
Spring Boot Actuator	Health and info endpoints
Springdoc OpenAPI 2.8.13	Swagger UI at /swagger-ui.html with an Authorize button
JUnit 5, Mockito, MockMvc	Unit and integration tests
H2 (test only)	In-memory database for the tests
Maven	Build and dependency management

5. System Architecture and Project Structure

The project uses a layered architecture. Each layer has one job, which keeps the code easy to maintain, test and extend.

```
Client (Postman / Swagger / Frontend)
        ↓
JWT Filter + Security Config (roles)
        ↓
Controller Layer
        ↓
Service Layer  ↔  AccessService (ownership checks)
        ↓
Repository Layer
        ↓
MySQL Database
```

Base package: com.agriculture.farm

Package	Purpose
auth	Registration, login, refresh tokens, AppUser, JwtService, JWT filter, AccessService (ownership), admin user management, first-admin setup.
config	SecurityConfig, OpenApiConfig (Authorize button), AuditConfig and SchedulerConfig.
controller	REST endpoints for farmers, farms, fields, crops, activities, expenses, harvests and reports.
service / service.impl	Business logic. Every service checks ownership through AccessService. ProfitService builds the profit report.
repository	Database operations using Spring Data JPA, including owner-scoped queries.
entity	JPA entities mapped to tables, plus status/type enums.
dto	Requests (input records) and Responses (output records).
mapper	EntityMapper converts entity to response DTO.
exception	ResourceNotFoundException, BadRequestException and GlobalExceptionHandler (400, 401, 403, 404, 409, 500).
src/test	JwtServiceTest, ProfitServiceTest and FarmApiIntegrationTest.

6. Main Entities and Relationships

Entity (table)	Purpose
AppUser (app_users)	Login account: name, unique email, BCrypt password hash, role (FARMER or ADMIN), active flag, linked Farmer profile, created time.
RefreshToken (refresh_tokens)	Stored refresh token: SHA-256 hash, owner, expiry time, revoked flag.
Farmer (farmers)	Farmer profile: name, unique email, phone, address, active flag, created/updated time.
Farm (farms)	A farm owned by a farmer: name, total area, status (ACTIVE / INACTIVE).
Field (fields)	A plot inside a farm: name, area, soil type, status (AVAILABLE / PLANTED / FALLOW).
Crop (crops)	A crop grown in a field: name, variety, planting and expected harvest dates, planned quantity, status.
Activity (activities)	Work done on a field: type, description, date, labour cost, quantity.
Expense (expenses)	Money spent on a farm: category, amount, date, description.
Harvest (harvests)	Harvest of a crop: date, quantity, unit, price per unit, total revenue.

Important relationships:

- One AppUser has one Farmer profile (created and linked automatically for FARMER accounts).
- One AppUser has many RefreshTokens.
- One Farmer has many Farms.
- One Farm has many Fields and many Expenses.
- One Field has many Crops and many Activities.
- One Crop has many Harvests.

- Deleting a parent removes its children (cascade ALL with orphanRemoval). Deleting a Farmer first unlinks its login account.

Status and type values:

Enum	Values
FarmStatus	ACTIVE, INACTIVE
FieldStatus	AVAILABLE, PLANTED, FALLOW
CropStatus	PLANNED, GROWING, HARVESTED, CANCELLED
ActivityType	PLOWING, SOWING, IRRIGATION, FERTILIZING, WEEDING, PEST_CONTROL, HARVESTING, OTHER
ExpenseCategory	SEED, FERTILIZER, PESTICIDE, LABOUR, EQUIPMENT, IRRIGATION, TRANSPORT, OTHER

7. Authentication and Security

Authentication uses Spring Security, BCrypt and JSON Web Tokens. Passwords are stored as BCrypt hashes, never as plain text. The server keeps no session (SessionCreationPolicy.STATELESS) and CSRF is disabled because the API uses Bearer tokens.

```
Register  → email checked (login accounts and farmer profiles)
          → password hashed with BCrypt → AppUser saved (role FARMER)
          → Farmer profile created and linked
          → access token + refresh token returned

Login     → AuthenticationManager verifies email + password (deactivated accounts are refused)
          → access token (30 min) + refresh token (7 days) returned

Protected request
          → client sends  Authorization: Bearer <access token>
          → JwtAuthenticationFilter validates the token and loads the user from the database
          → role and active status are checked on every request
          → controller and service run, AccessService checks ownership

Refresh   → client sends the refresh token to POST /api/auth/refresh
          → the old refresh token is marked used, a new pair is returned

Logout    → POST /api/auth/logout revokes the refresh token
```

Refresh token safety:

- Only the SHA-256 hash of a refresh token is stored in the database.
- Each refresh token works once. If an already-used token is presented again, every token of that user is revoked.
- Deactivating a user revokes all of their refresh tokens, and their access token stops working on the next request.
- Expired tokens are deleted every night at 03:00.

Class	Role in authentication
AuthController	Exposes /api/auth/register, login, refresh and logout.
AuthService	Registers users, authenticates logins, refreshes tokens and builds the AuthResponse.
FarmerProfileLinker	Creates and links a Farmer profile for FARMER accounts.

Class	Role in authentication
RefreshTokenService	Issues, rotates, revokes and purges refresh tokens.
CustomUserDetailsService	Loads an AppUser by email; deactivated users are marked disabled.
JwtService	Creates tokens (HMAC-signed, subject = email) and validates signature and expiry.
JwtAuthenticationFilter	Runs before each request, reads the Bearer token and authenticates the user.
SecurityConfig	Defines BCrypt encoder, public endpoints, role rules and the filter chain.
AdminInitializer	Creates the first ADMIN account at startup from environment variables.

Secrets and configuration:

- JWT_SECRET (Base64 key of at least 256 bits), DB_PASSWORD and the admin credentials come from environment variables.
- The built-in development secret only works in the default profile and logs a warning.
- The prod profile has no default values, so the application refuses to start unless every secret is set. Swagger is disabled in prod.
- Token lifetimes: JWT_EXPIRATION_MS (default 30 minutes) and JWT_REFRESH_EXPIRATION_MS (default 7 days).

8. Authorization and Data Ownership

Access is decided in two steps. SecurityConfig checks the role for each URL, then AccessService checks that the record belongs to the logged-in farmer.

Operation	Access
POST /api/auth/register, login, refresh, logout	Public
Swagger UI (/swagger-ui.html, /v3/api-docs)	Public
Health check (/actuator/health)	Public
/api/admin/** (user management)	ADMIN only
GET /api/farmers (list all farmers)	ADMIN only
DELETE on any /api/** endpoint	ADMIN only
All other /api/** endpoints	FARMER or ADMIN, limited to own data for FARMER

Ownership rules for a FARMER:

- A farm belongs to the farmer's profile. Fields, crops, activities, expenses and harvests belong to the farm they sit under.
- Reading, updating or adding data under another farmer's farm returns 403 Forbidden.
- List and search endpoints return only the farmer's own records; reports and profit cover only their own farms.
- A farmer cannot move a record to another farmer's farm.
- A FARMER cannot create extra farmer profiles. The profile is created at registration; GET /api/farmers/me returns it.

- ADMIN is not limited by ownership.

Security responses are returned as JSON:

Situation	Status
Missing, invalid or expired token	401 Unauthorized
Wrong email or password, or deactivated account	401 Unauthorized
Invalid, used or expired refresh token	401 Unauthorized
Duplicate registration email	409 Conflict
FARMER calls an ADMIN-only endpoint or another farmer's record	403 Forbidden

9. Admin User Management

These endpoints are available to ADMIN only (/api/admin/users).

Method	Endpoint	Purpose
GET	/api/admin/users?page=0&size=10	List all login accounts
GET	/api/admin/users/{id}	Get one account
PATCH	/api/admin/users/{id}/role	Change role: body {"role":"ADMIN"} or {"role":"FARMER"}
PATCH	/api/admin/users/{id}/active	Deactivate or re-activate: body {"active":false}
PUT	/api/admin/users/{id}/farmer	Link the account to an existing farmer profile: body {"farmerId":5}

Rules:

- A role change takes effect immediately, because the role is read from the database on every request.
- Deactivating a user blocks login, refresh and the current access token.
- An admin cannot remove their own ADMIN role or deactivate their own account.
- A farmer profile can be linked to only one account.
- If a user is changed to FARMER and has no profile, one is created.

10. DTOs and Validations

DTOs (Data Transfer Objects) are Java records that separate the API model from the database entities. Request records carry validation annotations; response records return only safe, flat data (for example farmerId and farmerName instead of the whole Farmer object).

DTO	Purpose
RegisterRequest / LoginRequest	Name, email and password (8 to 72 characters) for registration; email and password for login.
RefreshRequest	refreshToken, used for refresh and logout.
AuthResponse	success, message, tokenType, token, refreshToken, userId, name, email, role, farmerId.

DTO	Purpose
RoleRequest / ActiveRequest / LinkFarmerRequest	Admin inputs: role, active flag, farmerId.
UserSummary	Admin view of a user: id, name, email, role, active, farmerId, createdAt.
FarmerRequest / FarmerResponse	Farmer input and safe output.
FarmRequest / FarmResponse	Farm input (with farmerId) and output (with farmerName).
FieldRequest / FieldResponse	Field input (with farmId) and output (with farmName).
CropRequest / CropResponse	Crop input (with fieldId) and output (with fieldName).
ActivityRequest / ActivityResponse	Activity input (with fieldId) and output.
ExpenseRequest / ExpenseResponse	Expense input (with farmId) and output.
HarvestRequest / HarvestResponse	Harvest input (with cropId) and output including totalRevenue.
ExpenseReport / HarvestReport	Totals and counts for a date range.
ProfitSummary / FarmProfit	Revenue, expense and profit in total and per farm.

Important validations and business rules:

- Duplicate farmer or login email is rejected (400 for farmers, 409 for registration).
- Registration is refused if a farmer profile with that email already exists; an admin must link it, so nobody can take over another farmer's data.
- Names, emails, phone and address have maximum lengths; email must be a valid format.
- Farm totalArea and Field area must be positive numbers.
- Parent IDs (farmerId, farmId, fieldId, cropId) are required and must exist (404) and belong to the user (403).
- Expense amount, labour cost, quantity and planned quantity cannot be negative; harvest quantity must be greater than zero.
- Activity type, activity date, expense category, expense date and harvest date are mandatory.
- Status fields are optional; defaults are ACTIVE (farm), AVAILABLE (field) and PLANNED (crop).
- In reports, the "from" date must not be after the "to" date.
- Validation errors are collected field by field into one readable message.

11. Business Logic and Calculations

Harvest revenue is calculated automatically before a harvest is saved or updated:

```
Total Revenue = Quantity × Price per Unit
```

Profit for a farm and date range (a season):

```
Profit = Total Harvest Revenue − Total Expenses
```

Other logic in the service layer:

- When a harvest is created for a crop, that crop's status is automatically set to HARVESTED.

- Expense report: total amount and number of expenses between two dates, for all own farms or for one farm.
- Harvest report: total quantity, total revenue and number of harvests between two dates.
- Profit report: revenue, expense and profit per farm and in total. Activity labour cost is not added, so labour entered both ways is not counted twice.
- Crop search: case-insensitive partial match on crop name; crops can also be listed by status.
- Listing endpoints support page, size, sortBy and direction (asc / desc).
- Services run inside transactions; read-only methods use read-only transactions.

12. REST API Documentation

Every data module (farmers, farms, fields, crops, activities, expenses, harvests) has the same CRUD endpoints. The table shows them once, with {resource} standing for the module. All endpoints except register, login, refresh and logout need a Bearer token.

Method	Endpoint	Purpose
POST	/api/auth/register	Register a user and receive tokens (201)
POST	/api/auth/login	Login and receive tokens
POST	/api/auth/refresh	Exchange a refresh token for new tokens
POST	/api/auth/logout	Revoke a refresh token
POST	/api/{resource}	Create a record
GET	/api/{resource}?page=0&size=10&sortBy=id&direction=asc	List own records with paging and sorting
GET	/api/{resource}/{id}	Get one record
PUT	/api/{resource}/{id}	Update a record
PATCH	/api/{resource}/{id}	Update a record (same body as PUT)
DELETE	/api/{resource}/{id}	Delete a record (ADMIN only)
GET	/api/farmers/me	Get the logged-in farmer's profile
GET	/api/crops/search?name={text}	Search crops by name
GET	/api/crops/status/{status}	List crops by status
GET	/api/expenses/report?from=&to=&farmId=	Expense total and count (farmId optional)
GET	/api/harvests/report?from=&to=	Harvest quantity, revenue and count
GET	/api/reports/profit?from=&to=&farmId=	Revenue, expense and profit (farmId optional)
GET/PATCH/PUT	/api/admin/users...	User management (ADMIN only, see section 9)

Resource names: farmers, farms, fields, crops, activities, expenses, harvests. Dates use the format YYYY-MM-DD. Listing all farmers (GET /api/farmers) is ADMIN only.

13. Complete Farm Management Flow

Records depend on their parents, so they must be created in this order. Use the ID returned by each create call in the next request.

1 Register an account, or log in, and copy the token. A Farmer profile is created automatically; its id is the farmerId in the response.

2 Create a Farm using your farmerId.

3 Create a Field using the farmId.

4 Create a Crop using the fieldId.

5 Record Activities (fieldId) and Expenses (farmId) as work and spending happen.

6 Record a Harvest using the cropId; the crop becomes HARVESTED and revenue is calculated.

7 View the expense report, harvest report and profit report for a date range.

8 When the access token expires, call /api/auth/refresh with the refresh token.

```
Farmer → Farm → Field → Crop → Harvest
          │       └──→ Activity
          └──→ Expense
```

14. Example Requests

Register

```
POST /api/auth/register
{
  "name": "Ravi Kumar",
  "email": "ravi@example.com",
  "password": "StrongPass123"
}
```

The response contains token, refreshToken, role (FARMER) and farmerId. Send Authorization: Bearer <token> on the requests below.

Create a farm

```
POST /api/farms
{ "name": "Green Valley Farm", "totalArea": 12.5, "farmerId": 1 }
```

Record a harvest

```
POST /api/harvests
{
  "harvestDate": "2026-10-01",
  "quantity": 500,
  "pricePerUnit": 22,
  "unit": "kg",
  "cropId": 1
}
```

This harvest returns totalRevenue = 11000 and marks crop 1 as HARVESTED.

Profit report

```
GET /api/reports/profit?from=2026-01-01&to=2026-12-31
```

Refresh the access token

```
POST /api/auth/refresh
{ "refreshToken": "<refresh token from login>" }
```

Promote a user (ADMIN token required)

```
PATCH /api/admin/users/3/role
{ "role": "ADMIN" }
```

15. Testing

Run all tests with mvn test. They use an in-memory H2 database, so MySQL is not needed.

Test class	What it checks
JwtServiceTest	Token creation, validation, wrong user, expired token and wrong signing key.
ProfitServiceTest	Profit = revenue − expense per farm and in total, missing values, invalid date range.
FarmApiIntegrationTest	Full API flow with MockMvc: 401 without a token, automatic farmer profile, one farmer cannot read or change another farmer's data, only ADMIN can delete, profit report, refresh token rotation and reuse, logout, admin promote and deactivate, admin cannot demote themselves.

Manual testing order in Postman or Swagger:

Step	Test
1	Register an account (201 Created, tokens returned)
2	Log in and copy the token
3	Set Authorization > Bearer Token (or click Authorize in Swagger and paste only the token)
4	GET /api/farmers/me to see your profile and farmerId
5	Create farm, field and crop
6	Add activity and expense
7	Add harvest
8	Get lists, search crops, view expense, harvest and profit reports
9	Register a second user and confirm the first user's data returns 403
10	Log in as ADMIN: delete a record, list users, promote or deactivate a user
11	Call /api/auth/refresh, then try the old refresh token again (401)

16. Common Errors

Error	Solution
401 Unauthorized	Log in again, or refresh the token, and send a valid Bearer token. Paste the token without quotation marks.
403 Forbidden	The endpoint is ADMIN only, or the record belongs to another farmer.
409 Conflict	The registration email is already used, or a farmer profile with that email exists (ask an admin to link it).
404 Not Found	The record or parent ID does not exist, or the URL is wrong.
400 Bad Request	A validation rule failed or the email already exists; read the message field.
Authorize button missing in Swagger	Use the updated OpenApiConfig, restart the app and refresh the page.
Port 8080 already in use	Stop the other process or change server.port.
MySQL connection error	Start MySQL, create the database and check DB_PASSWORD.
JWT key error at startup	JWT_SECRET must be a Base64-encoded key of at least 256 bits.
App will not start with the prod profile	Set DB_URL, DB_USERNAME, DB_PASSWORD and JWT_SECRET.

17. Running, Configuration and Upgrade

Setup: create the database (CREATE DATABASE agriculture_farm_db), set DB_PASSWORD, ADMIN_EMAIL and ADMIN_PASSWORD, then run mvn clean spring-boot:run. The app starts on port 8080.

Setting	Purpose
DB_PASSWORD	MySQL password (default profile)
ADMIN_EMAIL / ADMIN_PASSWORD	First admin account, created once at startup (password at least 8 characters)
JWT_SECRET	Base64 signing key, at least 256 bits
JWT_EXPIRATION_MS	Access token lifetime (default 30 minutes)
JWT_REFRESH_EXPIRATION_MS	Refresh token lifetime (default 7 days)
DB_URL / DB_USERNAME	Database connection, required in the prod profile

Production: run with the prod profile (mvn spring-boot:run -Dspring-boot.run.profiles=prod). It has no default secrets and disables Swagger.

Upgrading an existing database: tables update automatically. app_users gets the active and farmer_id columns and a refresh_tokens table is created. Existing FARMER accounts get a farmer profile linked at their next login. If a profile with the same email already exists, it is not linked automatically; an admin links it with PUT /api/admin/users/{id}/farmer.

18. Project Summary, Resolved Limitations and Future Ideas

AFMSystem is a complete Spring Boot backend that combines REST APIs, JPA/Hibernate, MySQL, validation, exception handling, JWT authentication with refresh tokens, role-based authorization, per-farmer data ownership, admin user management, automatic revenue calculation, profit and date-range reports, and automated tests.

Limitations of the first version and how they were solved:

Earlier limitation	Solution
Any logged-in user could read and change every record	Ownership checks in every service and owner-scoped queries; other farmers' records return 403 and lists show only own data.
Login account was not linked to the Farmer profile	AppUser has a one-to-one link to Farmer; the profile is created at registration.
First ADMIN only through environment variables, no way to promote	Admin endpoints to list users, change roles, activate/deactivate and link profiles.
Default JWT secret and DB password placeholder	Prod profile with no defaults, startup warning for the dev secret, Swagger disabled in prod.
No profit reports	GET /api/reports/profit per farm and in total.
No tests	JUnit, Mockito and MockMvc tests with an in-memory database.
No refresh tokens	Rotating, hashed refresh tokens with logout, reuse detection and nightly cleanup.

Ideas for future work:

- Email verification and password reset.
- Login rate limiting and account lockout after repeated failures.
- Let farmers delete their own records (currently ADMIN only) with an audit trail.
- Docker packaging and a CI pipeline that runs the tests.